

Exploration C - Notes

Introduction to Data Frames

Data frames are fundamental objects in the R programming language, and when performing statistical tests using R.

Data Frames

A data frame is a rectangular array of values where each column is a vector. The columns can be different data types. When using a data frame in the context of a statistical study, usually each row of the data frame will correspond to a different *observational unit*, and the columns will correspond to the *variables* in your study (i.e. the characteristics that you are recording about each of your observational units).

To create a data frame from scratch, we can create a set of vectors of the same length and use the `data.frame()` function to make a data frame object. Notice that the inputs of the `data.frame()` function are all of the vectors that will compose the columns of the data frame, and that these inputs should be separated by commas.

Note: the `data.frame()` function is only needed if we are making a data frame from scratch in R. Most often, we will import our data from another file (like a csv or spreadsheet). This will be discussed at the end of this Exploration.

```
# creating vectors to use as columns
age <- c(1, 8, 10, 30, 31)
gender <- c("Female", "Female", "Male", "Female", "Male")
married <- c(FALSE, FALSE, FALSE, TRUE, TRUE)

# creating the data frame
simpsons <- data.frame(age, gender, married)

# displaying the data frame
simpsons
```

	age	gender	married
1	1	Female	FALSE
2	8	Female	FALSE
3	10	Male	FALSE
4	30	Female	TRUE
5	31	Male	TRUE

You may have noticed after running the code above that the data frame includes column names; data frames *always* have column names. When we use vectors to create data frames like in the example above, then the name of the vector is automatically selected as the column name.

If we input a vector like `c(1, 2, 3, 4, 5)` as an argument in the `data.frame()` function instead of an variable name, then R will make up a column name, like in the following example.

```
df <- data.frame(c(1,2,3,4,5), c("a","b","c","d","e"))
df
```

	c.1..2..3..4..5.	c..a....b....c....d....e..
1	1	a
2	2	b
3	3	c
4	4	d
5	5	e

Generally, it is best practice to give your vectors variable names and use those as the inputs to `data.frame()` rather than using the vectors directly as the inputs for the function (as in the previous code block).

Data frames must always have column names, and often have row names too! We will discuss in upcoming sections how to change the row and column names of data frames explicitly.

Viewing a Data Frame

If you have given a variable name to a data frame, you can display the data frame in a code block by simply writing its name. Alternatively, you can view the entirety of a data frame in a nicely formatted way by clicking on the variable name of the data frame in the **Environment** pane.

Head and Tail

It can also be helpful when working with a large data frame to use the `head()` and `tail()` functions, which display with the first part or last part of the data frame (respectively). You just need to specify the data frame and how many lines you want to display. If you don't specify a number of lines, the default is 6 lines.

```
# Defaults to display first 6 rows of built in data frame iris
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
# Displays last 4 rows of built in data frame iris
tail(iris, n=4)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica

Names

As mentioned earlier, data frames will always have column names, and can also have row names. To see the row or column names of a data frame, we can use the `rownames()` and `colnames()` functions. As you can see in the next code block, if you do not assign row names to a data frame, R will automatically assign numbers to the rows.

```
rownames(simpsons)
```

```
[1] "1" "2" "3" "4" "5"
```

```
colnames(simpsons)
```

```
[1] "age"      "gender"   "married"
```

If we want to change these names we can use the same functions and the assignment operator `<-` to manually reassign the names.

```
# Add names of characters to rows
rownames(simpsons) <- c("Maggie", "Lisa", "Bart", "Marge", "Homer")
simpsons
```

	age	gender	married
Maggie	1	Female	FALSE
Lisa	8	Female	FALSE
Bart	10	Male	FALSE
Marge	30	Female	TRUE
Homer	31	Male	TRUE

```
# Change names of columns to capitals
colnames(simpsons) <- c("Age", "Gender", "Married")
simpsons
```

	Age	Gender	Married
Maggie	1	Female	FALSE
Lisa	8	Female	FALSE
Bart	10	Male	FALSE
Marge	30	Female	TRUE
Homer	31	Male	TRUE

Dimensions

Data frames are two-dimensional, (as opposed to vectors/factors which are one-dimensional). So rather than using the `length()` function to determine the number of elements like we used for vectors/factors, there are three functions we can use, depending on what dimension you want to know about.

- The function `dim()` will give you a vector containing both the number of rows and the number of columns in the data frame (with the number of rows displayed first).
- The function `nrow()` will give you only the number of rows.
- The function `ncol()` will give you the only number of columns.

```
dim(simpsons)
```

```
[1] 5 3
```

```
nrow(simpsons)
```

```
[1] 5
```

```
ncol(simpsons)
```

```
[1] 3
```

Accessing Elements

Often times we will have a data frame with many rows and columns, but will want access to a smaller subsets of the data frame (e.g. certain rows only, or certain columns only). With vectors/factors we used brackets `[]` to access certain elements. With data frames, there are two options.

- If you want to access a single column of a data frame, you can use the `$` operator. The format for using this operator is as follows: `df$colName`, where `df` would be the name of the data frame and `colName` would be the name of the column that you are trying to access. Notice that there are *no quotes needed for the column name in this format*.
- To access either rows or columns or some combination of the two, you will need to use brackets with a comma `[,]`. The format for using this operator is as follows: `df[row, column]`, where `df` is the name of the data frame, and `row` and `column` can be an method of indexing (e.g. numbers or names). Note that if you use names in this method, *quotes are necessary around the name*. Additionally note that if you only want to access a whole row or a whole column, you can omit either of the inputs: `df[row,]` gives whole rows and `df[,column]` gives whole columns.

Below are some examples of ways to access rows and columns from a data frame in various combinations. **Note:** if you read these notes in RStudio, you may notice that some of the methods of accessing elements give you a data frame while some of the methods give you vectors instead.

```
# Get a the first row  
simpsons[1,]
```

```

      Age Gender Married
Maggie  1 Female   FALSE

```

```

# Get the second column
simpsons[,2]

```

```

[1] "Female" "Female" "Male"  "Female" "Male"

```

```

# Get the element in the first row, second column
simpsons[1,2]

```

```

[1] "Female"

```

```

# Get the element in the first row, second column using column name
simpsons[1,"Gender"]

```

```

[1] "Female"

```

```

# Get the second column using $ notation
simpsons$Gender

```

```

[1] "Female" "Female" "Male"  "Female" "Male"

```

```

# Accessing the first and second rows at once using c()
simpsons[c(1,2),]

```

```

      Age Gender Married
Maggie  1 Female   FALSE
Lisa    8 Female   FALSE

```

```

# Accessing the first and second columns at once using c() and column names
simpsons[, c("Age", "Gender")]

```

```

      Age Gender
Maggie  1 Female
Lisa    8 Female
Bart   10  Male
Marge  30 Female
Homer  31  Male

```

Note that using the `$` notation gives you a vector and that you can then use the `[]` without the comma if you wanted to access a certain element in that vector.

```
# getting the age column using $
simpsons$Age
```

```
[1] 1 8 10 30 31
```

```
# getting the second element of the age column using $ and []
simpsons$Age[2]
```

```
[1] 8
```

Subset

The `subset()` function is an alternative method to access data and is especially helpful when you want to access data that meets certain conditional criteria. For example, if we want to take all the rows of a data frame which have a particular value in a certain column, we can use the `subset()` function.

The subset function requires two inputs (separated by commas):

- The first input should be the name of the data frame that you are pulling the data from.
- The second input should be some logical expression, perhaps using both logical and relational operators, that describes the values that you want to isolate from the data frame. Note that this function has the special feature that you don't need to specify the name of the data frame first (i.e. instead of `subset(simpsons, simpsons$Age > 10)` you can do `subset(simpsons, Age > 10)`).

The output of this function will be a new data frame satisfying the conditions that you specify.

Consider the following data frame.

```
# defining the columns
group <- c("G1", "G2", "G1", "G1", "G2")
age <- c(35, 30, 31, 28, 40)
height <- c(65, 70, 60, 72, 68)
pets <- c(TRUE, TRUE, FALSE, FALSE, TRUE)

# creating the data frame
mydata <- data.frame(group, age, height, pets)
mydata
```

	group	age	height	pets
1	G1	35	65	TRUE
2	G2	30	70	TRUE
3	G1	31	60	FALSE
4	G1	28	72	FALSE
5	G2	40	68	TRUE

In the following code, we use the `subset()` function to isolate only the rows in `mydata` such that the `group` attribute for that row is equal to "G1".

```
group1 <- subset(mydata, group == "G1")
group1
```

	group	age	height	pets
1	G1	35	65	TRUE
3	G1	31	60	FALSE
4	G1	28	72	FALSE

To subset by all values which are NOT equal to a condition we can use the logical operator `!=`. In the following code, `group2` holds all of the rows of `mydata` for which the group is not equal to "G1".

```
group2 <- subset(mydata, group != "G1")
group2
```

	group	age	height	pets
2	G2	30	70	TRUE
5	G2	40	68	TRUE

In general, we can use any logical and relational operators we want in this second argument (see Exploration A if you need a refresher on logical and relational operators). For instance, I could extract all the rows where the group is "G2" OR the age is greater than or equal to 30.

```
g2_over30 <- subset(mydata, group == "G2" | age >= 30)
g2_over30
```

	group	age	height	pets
1	G1	35	65	TRUE
2	G2	30	70	TRUE
3	G1	31	60	FALSE
5	G2	40	68	TRUE

In the next Exploration, we will see many more interesting ways of accessing data that are similar to the `subset()` function.

Summarizing Data using `summary()` or `table()`

The `summary()` function is a powerful command that gives you some summary statistics about the variables in the data frame.

```
summary(mydata)
```

	group	age	height	pets
Length	:5	Min. :28.0	Min. :60	Mode :logical
N.unique	:2	1st Qu.:30.0	1st Qu.:65	FALSE:2
N.blank	:0	Median :31.0	Median :68	TRUE :3
Min.nchar	:2	Mean :32.8	Mean :67	
Max.nchar	:2	3rd Qu.:35.0	3rd Qu.:70	
		Max. :40.0	Max. :72	

The summary statistics are listed below the names of the variables. Since `pets` is a logical variable, R gives you the frequencies of each unique value. In this example there are three values of `TRUE` and two values of `FALSE`. Since `age` and `weight` are numeric, R computes and returns the minimum, 1st quartile (25th percentile), median, mean, 3rd quartile (75th percentile), and maximum values. If you have many data values, this is a quick way to get a feel for how the data are distributed.

Just like we did for vectors, we can also use the `table()` to cross-tabulate categorical data. Let's create a frequency table for the `group` column of the `mydata` data frame.

```
table(mydata$group)
```

```
G1 G2
3  2
```

We can also create two-way frequency tables, like we did when working with factors. Here we create a two-way frequency table of pet status for both groups.

```
table(mydata$group, mydata$pets)
```

	FALSE	TRUE
G1	2	1
G2	0	2

Adding Columns

One can add a new variable (column) to a data frame by defining a new variable and assigning values to it. Below we add a `weight` variable to the data frame `mydata`.

```
# creating the weights vector
wgths <- c(169, 161, 149, 165, 155)

# adding a weights column that will have the name weight
mydata$weight <- wgths

# displaying the data frame
mydata
```

	group	age	height	pets	weight
1	G1	35	65	TRUE	169
2	G2	30	70	TRUE	161
3	G1	31	60	FALSE	149
4	G1	28	72	FALSE	165
5	G2	40	68	TRUE	155

We can also add a new column using the `cbind()` function (column bind function).

```
# adding the wgths column
alt_mydata <- cbind(mydata, wgths)

# displaying the data frame
alt_mydata
```

	group	age	height	pets	weight	wgths
1	G1	35	65	TRUE	169	169
2	G2	30	70	TRUE	161	161
3	G1	31	60	FALSE	149	149
4	G1	28	72	FALSE	165	165
5	G2	40	68	TRUE	155	155

Note: if you run the command to add a column multiple times, it will add multiple rows. Keep in mind though that when you render the document, the compiler starts fresh as if you have never run the code and will only run each line of code once. See the beginning of Exploration B if you want a reminder of how to reset the environment so that you can see more clearly what will happen when you render your document.

Adding Rows

Rows can be added to a data frame using the `rbind()` (row bind) function. Because our columns have different data types, we must create something called a list, which allows different data types, and then add it as a new row.

```
# creating a list with values for the new row
newobs <- list("G1", 23, 62, FALSE, 160)

# adding the new row to the data frame
newdata <- rbind(mydata, newobs)

# displaying the data frame
newdata
```

	group	age	height	pets	weight
1	G1	35	65	TRUE	169
2	G2	30	70	TRUE	161
3	G1	31	60	FALSE	149
4	G1	28	72	FALSE	165
5	G2	40	68	TRUE	155
6	G1	23	62	FALSE	160

We can also use `rbind()` to append one data frame to another. We can do this with the variables `group1` and `group2` created above.

```
group1
```

	group	age	height	pets
1	G1	35	65	TRUE
3	G1	31	60	FALSE
4	G1	28	72	FALSE

```
group2
```

	group	age	height	pets
2	G2	30	70	TRUE
5	G2	40	68	TRUE

```
rbind(group1, group2)
```

	group	age	height	pets
1	G1	35	65	TRUE
3	G1	31	60	FALSE
4	G1	28	72	FALSE
2	G2	30	70	TRUE
5	G2	40	68	TRUE

NA values

Remember, if we have a missing value, or a blank value, we can use the object NA to indicate the lack of a value.

```
fav_color <- c("Red", NA, "Purple", NA, "Red")  
  
mydata <- cbind(mydata, fav_color)  
mydata
```

	group	age	height	pets	weight	fav_color
1	G1	35	65	TRUE	169	Red
2	G2	30	70	TRUE	161	<NA>
3	G1	31	60	FALSE	149	Purple
4	G1	28	72	FALSE	165	<NA>
5	G2	40	68	TRUE	155	Red

Sometimes certain functions that we might want to apply to a data frame will not work properly if a data frame includes NA values. We can check if a data frame contains NA values using the `is.na()` function.

```
is.na(mydata)
```

	group	age	height	pets	weight	fav_color
[1,]	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
[2,]	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
[3,]	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
[4,]	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
[5,]	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE

If we want to remove all rows that contain any NA values, we can use `na.omit()`. The output of this function will be a new data frame excluding all rows that contain an NA value in any column.

```
na.omit(mydata)
```

	group	age	height	pets	weight	fav_color
1	G1	35	65	TRUE	169	Red
3	G1	31	60	FALSE	149	Purple
5	G2	40	68	TRUE	155	Red

Alternatively, if you only want to remove rows that contain NA in a certain column, you can use the `is.na()` function, the not operator `!`, and `subset` function. In particular, this code is saying: take only the rows of `mydata` for which the `fav_color` value of that row is *not* an NA value. In other words, this code will provide you with a new data frame excluding any rows for which the specified column contains an NA value.

```
# keep only rows where the fav_color column is not NA
subset(mydata, !is.na(fav_color))
```

	group	age	height	pets	weight	fav_color
1	G1	35	65	TRUE	169	Red
3	G1	31	60	FALSE	149	Purple
5	G2	40	68	TRUE	155	Red

NULL

We can remove a specific column (variable) from a data set by setting it equal to the special value `NULL`.

```
# removing the fav_color column
mydata$fav_color <- NULL
mydata
```

	group	age	height	pets	weight
1	G1	35	65	TRUE	169
2	G2	30	70	TRUE	161
3	G1	31	60	FALSE	149
4	G1	28	72	FALSE	165
5	G2	40	68	TRUE	155

Importing Data Sets

Built-In Data

There are various objects that are built-in to R. For example `pi` returns the value `3.141593`. Similarly, there are built-in data sets that are ready to be loaded and used at a moments notice.

To see a list of built in data sets type the following into the console: `data()`. This should open a new window in the top left pane that displays a list of all of the available data sets built-in to R.

These data sets can be used even if they are not listed in the global environment. For example, if you would like to load the data set `cars` in the global environment, run the following command:

```
data("cars")
```

You should then see the `cars` data frame listed in the **Environment** pane in the top right corner.

You can also simply type the name of a built-in data set in a code block and run the code block to display the data.

```
women
```

	height	weight
1	58	115
2	59	117
3	60	120
4	61	123
5	62	126
6	63	129
7	64	132
8	65	135
9	66	139
10	67	142
11	68	146
12	69	150
13	70	154
14	71	159
15	72	164

Importing From Your Computer

Although built-in data sets are convenient, in practice we often need to load our own data sets. We load our own data sets by using a function specifically designed for the file type of interest. This function usually uses the file path location as an argument. This can be done in many different ways. In this class, we will typically import our data from a “csv” (or “comma separated value”) file. Any spreadsheet (like a Google Sheet or Excel Spreadsheet) can be saved or converted to a csv file.

Below are the steps that you can take to import a csv file as a data frame in R. The `InsectData.csv` will be used as an example. (Steps 1 and 2 have already been completed in this example.)

- 1) Download the file `InsectData.csv` from **Canvas**.
- 2) In the **Files** tab (lower right panel), select **Upload** and then **Choose file**. Find the `InsectData.csv` file (its probably in your downloads folder), and open that file. The file `InsectData.csv` should now be listed in the list of files in the **File** tab.
- 3) In a code block, type the following line of code. This line of code can be included in any code block in the document as long as these it appears before your first use of the data frame.

```
InsectData <- read.csv("InsectData.csv")
```

You should now see the name of the data set listed as a variable in the environment.

Import From Online

We can also download data sets from online in a variety of different ways. Below is one option. With this method we are using the same `InsectData.csv` file, but it has been posted online. We feed the url of where the data set has been posted into the `read.csv()` function in order to open the file.

```
the_url <- "https://raw.githubusercontent.com/rpkgarcia/LearnRBook/main/data_sets/InsectData.csv"
the_data <- read.csv(the_url)
```

Notice that in each of these scenarios, *the data is imported as a data frame*, and the variable name of that data frame is the name of the csv file.

Once your data has been imported, you can use it just like any data frame.

```
colnames(InsectData)
```

```
[1] "wt"      "length" "sex"
```

```
InsectData[1,2]
```

```
[1] 18
```

```
subset(InsectData, wt > 2)
```

	wt	length	sex
1	2.58	18	m
2	3.12	21	f
3	2.09	16	m
4	3.21	22	f
6	2.75	19	m
7	2.63	15	m
8	2.99	20	m
10	3.02	21	f